PRODUCTION MASTERPLAN

# Docker for Developers: Zero to Production

Complete End-to-End Execution Blueprint & Pedagogical Architecture for YouTube

| TARGET FORMAT | TARGET AUDIENCE | RETENTION TARGET | PRODUCTION CYCLE |
|---|---|---|---|
| **120–150m** | **Full-Stack / DevOps** | **42%+ AVD** | **4 Weeks** |
| Single Mega-Course or 6-Part Series | Beginner to Intermediate devs | High practical code density | Research to release & promo |

## 4-Week Video Production Timeline

PHASED EXECUTION

| Phase | Focus Area | Core Deliverables & Actions | Status / Milestones |
|---|---|---|---|
| **Week 1** | Course Scoping & Scripting | Outline visual script, script real-world scenario (Node.js + Postgres), draft 10 terminal code snippets, build clean sandbox repository. | Curriculum Locked |
| **Week 2** | Assets, Slides & Lab Build | Create high-contrast terminal theme (Catppuccin/Dracula, 20pt+ font), create animated architecture slides, verify cross-platform Docker Desktop behavior. | Test Repo Ready |
| **Week 3** | Studio Recording | Batch record screencasts in 4K/60fps with OBS; record dual-track audio (Rode/Shure) with zero noise floor; record facecam reaction/explainer shots. | Raw Footage Done |
| **Week 4** | Post-Prod & Packaging | Insert zoom-in transitions on code snippets, sound effects on outputs, design 3 high-CTR thumbnails (85%+ contrast), generate pinned GitHub repository. | Published & Promoted |

## Hardware, Software & Screencast Rig Setup

TECHNICAL STACK

### Visuals & Terminal Typography

- **Resolution:** 1440p (2560x1440) or 4K scaled canvas to prevent blurring on mobile.
- **Font:** JetBrains Mono or Fira Code at 18–22pt with ligatures enabled.
- **Terminal Prompt:** Minimal Starship prompt hiding full absolute paths to avoid clutter.
- **B-Roll & Overlays:** Clear animated diagrams showing host vs. container layers.

### YouTube Optimization & Growth Hook

- **The First 60 Seconds:** "It works on my machine is dead." Show broken app vs `docker compose up` fix.
- **Chapter Timestamps:** Exact labels matching high-volume search queries.
- **Companion Kit:** Public GitHub repo with branch-per-module and cheat sheet.
- **Retention Triggers:** On-screen progress pills indicating current chapter progress.

**Guiding Pedagogical Rule: "Concept Before Command"**

Never run a CLI command without first illustrating the exact state change happening under the hood (e.g., explaining layer caching before writing `RUN npm install`). This drastically reduces cognitive overload and drives longer watch times.

## Curriculum Breakdown: Modules 1 to 4

FOUNDATIONAL TO INTERMEDIATE

### Module 1: The Core Mental Model & Virtualization Shift

Estimated: 18 Mins

- **The Problem Statement:** Dependency drift, operating system conflicts, and why virtual machines (VMs) waste memory and hypervisor overhead.
- **Containers vs Virtual Machines:** Visual comparison of Guest OS + Hypervisor vs Shared Linux Kernel + Namespaces + Cgroups.
- **Docker Architecture:** Docker Client (CLI), Docker Daemon (dockerd), Containerd runc, and Docker Hub image registry.
- **Setup:** Clean installation of Docker Desktop on Mac/Windows (WSL2 integration) and bare-metal engine on Linux (Ubuntu).

### Module 2: Essential Container Lifecycle & CLI Operations

Estimated: 22 Mins

- **Running First Container:** `docker run -d --name web -p 8080:80 nginx:alpine` — dissecting each flag and parameter.
- **Process Inspection & Logs:** Real-time streaming with `docker logs -f`, inspecting metrics with `docker stats`, and inspecting JSON data via `docker inspect`.
- **Interactive Debugging:** Shell execution via `docker exec -it web sh`; troubleshooting file changes inside living containers.
- **Housekeeping & Hygiene:** Clean deletion patterns: `docker stop`, `docker rm`, `docker rmi`, and automated garbage collection with `docker system prune -a --volumes`.

### Module 3: Authoring Production-Grade Dockerfiles

Estimated: 25 Mins

- **Anatomy of a Dockerfile:** Instruction set deep-dive: `FROM`, `WORKDIR`, `COPY`, `RUN`, `ENV`, `EXPOSE`, `ENTRYPOINT` vs `CMD`.
- **Layer Caching Mechanics:** Ordering instructions to maximize build cache hits (copying package.json before entire source tree).
- **Exclusion Discipline:** Creating robust `.dockerignore` files to block `node_modules`, logs, secrets, and git history.
- **Multi-Stage Builds:** Compiling in a heavy build environment and extracting artifacts into a slim distroless/alpine runtime image (slashing image sizes from 1.2GB to 85MB).

### Module 4: Storage Persistency & Dynamic Networking

Estimated: 20 Mins

- **Ephemeral Nature of Containers:** Demonstrating data loss on container teardown and the need for persistent storage.
- **Bind Mounts vs Named Volumes:** Bind mounts for instantaneous host-to-container hot reloading in development vs named volumes for high-performance production databases.
- **Docker Networking:** Default bridge, host mode, and custom user-defined bridge networks.
- **Embedded DNS:** How containers automatically discover each other by container name without hardcoded IP addresses.

> **Creator Tip: Live Debugging Creates Value**
> Intentionally provoke a `Port already allocated` or `CrashLoopBackOff` error during Module 2 and Module 4. Walking through terminal error logs builds higher trust than a sanitized tutorial where nothing ever breaks.

## Curriculum Breakdown: Modules 5 & 6

PRODUCTION ENGINEERING

### Module 5: Multi-Container Orchestration with Docker Compose

Estimated: 30 Mins

- **Why Compose?:** Moving beyond unwieldy 100-character terminal scripts into declarative, version-controlled `docker-compose.yml`.
- **Real-World Fullstack Architecture:** Orchestrating a 3-tier application: React frontend, Node.js API, and PostgreSQL database.
- **Service Dependencies:** Using `depends_on` paired with `condition: service_healthy` to prevent API crashes before database migration finishes.
- **Environment Isolation:** Managing secrets and configs using `.env` files and Compose variable substitution.

### Module 6: Security, Image Publishing & CI/CD Integration

Estimated: 25 Mins

- **Container Hardening:** Enforcing non-root user execution (`USER appuser`) and read-only filesystems.
- **Vulnerability Scanning:** Scanning base images for CVEs using `docker scout` and Trivy before publishing.
- **Registry Distribution:** Tagging strategies (semver vs sha hashes), authenticating, and publishing to Docker Hub / GHCR.
- **Automated Pipeline:** Writing a GitHub Actions workflow to build, lint, scan, and push Docker images automatically on git merge.

## YouTube Publishing & Promotion Checklist

POST-PRODUCTION

### Packaging & SEO Metadata

• **Titles:** "Docker in 2 Hours: Zero to Production", "The Only Docker Tutorial You Need (2026)".

• **Thumbnail Strategy:** High contrast, minimalist Whale icon, facial expression, 3-word title (e.g., "STOP USING VMs").

• **Description:** Full timestamp table, links to starter and finished repo branches, PDF cheat sheet download.

### Community & Lead Generation

• **Pinned Comment:** Challenge question: "What stack are you containerizing first?"

• **Shorts/Reels Repurposing:** Cut 4 high-value clips ("3 Dockerfile mistakes", "Multi-stage builds in 60s").

• **Distribution:** Share key architectural diagrams on X/LinkedIn and Reddit (r/docker, r/webdev).

**Final Action Item: The Starter Repository**

Ensure the companion repository has a `starter` branch with uncontainerized boilerplate code, and a `solution` branch for each chapter. This enables viewers to code along without frustration and drives high star engagement on GitHub.